Actividad Python - Entrada y Salida

Guía de ejecución y funcionamiento

Cómo ejecutar los ejercicios

1. Instala Python 3. En Windows, durante la instalación conviene activar la opción para agregar Python al PATH.
2. Abre una terminal en la carpeta phyton/entrada_salida. Cada archivo es un programa independiente.
3. Comprueba la instalación con python --version; en algunos equipos Windows también puedes usar py --version.
4. Ejecuta un ejercicio, por ejemplo: python banner.py. Escribe los datos cuando el programa los solicite.

También puedes abrir un archivo en VS Code, IDLE u otro editor de Python y ejecutarlo desde allí.

Ejercicios incluidos

01. banner.py	02. calculadora.py
03. calificaciones.py	04. control.py
05. distancia.py	06. divisas.py
07. formatear.py	08. generador.py
09. propina.py	10. validador.py

Las entradas y salidas de ejemplo se comprobaron ejecutando cada programa. Las tasas de divisas son valores fijos de ejemplo. Para que los listados se lean con la fuente incrustada, los emojis de confirmación y error se representan como ✓ y ✗.

1. Banner de bienvenida

Requerimiento de la actividad: Solicita el nombre del aprendiz y el programa de formación y los presenta dentro de un recuadro.

Archivo: banner.py

```
nombre = input("Ingrese el nombre del aprendiz: ").strip()
programa = input("Ingrese el nombre del programa de formación: ").strip()

ancho = 50  # ancho total del recuadro

print("*" * ancho)
print(f"*{'BIENVENIDA'.center(ancho - 2)}*")
print(f"*{('Aprendiz: ' + nombre.upper()).center(ancho - 2)}*")
print(f"*{('Programa: ' + programa).center(ancho - 2)}*")
print("*" * ancho)
```

Cómo funciona: strip() elimina espacios externos; upper() convierte el nombre a mayúsculas y center() centra cada renglón dentro del ancho definido.

Cómo ejecutar: python banner.py

Datos de ejemplo: Alejandro; Análisis de datos

Salida de referencia

```
**************************************************
*                   BIENVENIDA                   *
*              Aprendiz: ALEJANDRO               *
*          Programa: Análisis de datos           *
**************************************************
```

Conceptos clave: input(), cadenas, strip(), upper(), center(), f-strings y repetición de caracteres.

2. Cálculo de depreciación anual

Requerimiento de la actividad: Calcula la depreciación anual dividiendo el valor de compra entre los años de vida útil.

Archivo: calculadora.py

```
valor_compra = float(input("Ingrese el valor de compra del equipo informático: "))
vida_util = int(input("Ingrese los años de vida útil: "))

if vida_util &lt;= 0:
    print("Los años de vida útil deben ser mayores que 0.")
else:
    depreciacion_anual = valor_compra / vida_util

    # Formato de moneda colombiana
    valor_formateado = f"{depreciacion_anual:,.0f}"
    valor_formateado = valor_formateado.replace(",", "X")
    valor_formateado = valor_formateado.replace(".", ",")
    valor_formateado = valor_formateado.replace("X", ".")

    print(f"Depreciación anual: {valor_formateado} COP")
```

Cómo funciona: Valida que la vida útil sea mayor que cero. Divide el valor entre los años y adapta los separadores para mostrar el resultado en pesos colombianos.

Cómo ejecutar: python calculadora.py

Datos de ejemplo: Valor de compra: 1000000; vida útil: 5 años

Salida de referencia

```
Depreciación anual: 200.000 COP
```

Conceptos clave: float, int, input(), condicional if/else, división y replace().

3. Validación de calificaciones

Requerimiento de la actividad: Lee una nota entre 0.0 y 5.0, acepta coma decimal y comunica si el valor es numérico y está dentro del rango.

Archivo: calificaciones.py

```
entrada = input("Ingrese una nota (0.0 a 5.0): ").replace(",", ".")

try:
    nota = float(entrada)

    if nota &lt; 0.0 or nota &gt; 5.0:
        print(f"✗ Error de rango: la nota {nota} está fuera del rango permitido (0.0 a 5.0).")
    else:
        print(f"✓ Nota registrada correctamente: {nota:.1f}")

except ValueError:
    print(f"✗ Error de tipo: '{entrada}' no es un número válido.")
```

Cómo funciona: replace() cambia la coma por punto antes de convertir con float(). ValueError se captura si la entrada no es numérica; después se comprueban los límites.

Cómo ejecutar: python calificaciones.py

Datos de ejemplo: 4,5

Salida de referencia

```
Nota registrada correctamente: 4.5
(El programa antepone un símbolo de confirmación.)
```

Conceptos clave: Cadenas, replace(), float, try/except, ValueError y operadores relacionales.

4. Control de acceso por edad y documento

Requerimiento de la actividad: Permite el acceso únicamente a una persona de 18 años o más que indique tener documento físico.

Archivo: control.py

```
edad = int(input("Ingrese su edad: "))
documento = input("¿Cuenta con documento físico? (si/no): ").strip().lower()

if documento == "si" or documento == "sí":
    documento_fisico = True
else:
    documento_fisico = False

if edad &gt;= 18 and documento_fisico:
    print("Acceso permitido.")
else:
    print("Acceso no permitido.")
```

Cómo funciona: La respuesta se limpia y pasa a minúsculas, luego se convierte en booleano y se combina con la comparación de edad mediante and.

Cómo ejecutar: python control.py

Datos de ejemplo: Edad: 20; documento físico: si

Salida de referencia

```
Acceso permitido.
```

Conceptos clave: int, strip(), lower(), booleanos, comparaciones y operadores and/or.

5. Tiempo estimado de viaje

Requerimiento de la actividad: Calcula cuánto tarda un viaje a partir de su distancia en kilómetros y velocidad promedio en km/h.

Archivo: distancia.py

```
distancia = float(input("Distancia del viaje (km): "))
velocidad = float(input("Velocidad promedio (km/h): "))

if velocidad &lt;= 0:
    print("Error: La velocidad debe ser mayor que cero.")
elif distancia &lt; 0:
    print("Error: La distancia no puede ser negativa.")
else:
    tiempo_total_h = distancia / velocidad      # horas en decimal
    horas   = int(tiempo_total_h)               # parte entera
    minutos = round((tiempo_total_h - horas) * 60)  # residuo * 60

    # Ajuste por redondeo: si minutos da 60, hay que sumar 1 hora
    if minutos == 60:
        horas += 1
        minutos = 0

    print("\n--- TIEMPO ESTIMADO DE VIAJE ---")
    print(f"Distancia:  {distancia:,.2f} km")
    print(f"Velocidad:  {velocidad:,.2f} km/h")
    print(f"Tiempo:     {horas} horas y {minutos} minutos")
```

Cómo funciona: La división obtiene las horas decimales; int() toma las horas completas y el residuo se convierte a minutos. Valida valores no permitidos y corrige un redondeo de 60 minutos.

Cómo ejecutar: python distancia.py

Datos de ejemplo: Distancia: 100 km; velocidad: 80 km/h

Salida de referencia

```
--- TIEMPO ESTIMADO DE VIAJE ---
Distancia:  100.00 km
Velocidad:  80.00 km/h
Tiempo:     1 horas y 15 minutos
```

Conceptos clave: float, división, int, round(), formato numérico y condicionales.

6. Conversión de pesos a otras divisas

Requerimiento de la actividad: Convierte un monto en pesos colombianos a dólares estadounidenses y euros.

Archivo: divisas.py

```
# Tasas de cambio
TASA_USD = 4000
TASA_EUR = 4300

# --- Entrada del usuario (solo el monto) ---
monto_cop = float(input("Monto en Pesos Colombianos (COP): "))

# --- Validación ---
if monto_cop &lt; 0:
    print("Error: El monto no puede ser negativo.")
else:
    dolar = monto_cop / TASA_USD
    euro = monto_cop / TASA_EUR

    print("\n--- CONVERSIÓN DE DIVISAS ---")
    print(f"Monto en COP:       ${monto_cop:&gt;15,.2f}")
    print(f"Equivalente en USD: ${dolar:&gt;15,.2f}")
    print(f"Equivalente en EUR: ${euro:&gt;15,.2f}")
```

Cómo funciona: Divide el monto entre las tasas declaradas en el archivo: 4.000 COP por USD y 4.300 COP por EUR. Son valores fijos de ejemplo, no tasas en tiempo real.

Cómo ejecutar: python divisas.py

Datos de ejemplo: 100000 COP

Salida de referencia

```
--- CONVERSIÓN DE DIVISAS ---
Monto en COP:       $     100,000.00
Equivalente en USD: $          25.00
Equivalente en EUR: $          23.26
```

Conceptos clave: Constantes, float, división, validación de negativos y formato numérico.

7. Ticket de supermercado

Requerimiento de la actividad: Registra tres productos con cantidad y precio, calcula los subtotales y muestra el total general en una tabla.

Archivo: formatear.py

```
# Lista vacía donde guardaremos cada producto
productos = []

# Bucle: se repite 3 veces
for i in range(1, 4):
    print(f"\n--- Producto {i} ---")
    nombre   = input("Nombre: ").strip()
    cantidad = int(input("Cantidad: "))
    precio   = int(input("Precio unitario: $"))
    subtotal = cantidad * precio

    # Guardamos los 4 valores como una tupla dentro de la lista
    productos.append((nombre, cantidad, precio, subtotal))

# Calcular total general recorriendo la lista
total_general = 0
for p in productos:
    total_general += p[3]   # p[3] es el subtotal

# Imprimir la tabla
print("\n" + "=" * 62)
print(f"{'PRODUCTO':&lt;20} {'CANTIDAD':&gt;2} {'PRECIO':&gt;13} {'SUBTOTAL':&gt;16}")
print("-" * 62)

for nombre, cantidad, precio, subtotal in productos:
    print(f"{nombre:&lt;20} {cantidad:&gt;6} ${precio:&gt;12,.0f} ${subtotal:&gt;15,.0f}")

print("-" * 62)
print(f"{'TOTAL GENERAL':&lt;20} {'':&gt;6} {'':&gt;13} ${total_general:&gt;15,.0f}")
print("=" * 62)
```

Cómo funciona: Cada producto se guarda como tupla dentro de una lista. Después se suman los subtotales y se alinean las columnas con especificadores de ancho en f-strings.

Cómo ejecutar: python formatear.py

Datos de ejemplo: Arroz: 2 unidades a 5000; Leche: 1 a 4000; Pan: 3 a 1500

Salida de referencia

```
==============================================================
PRODUCTO             CANTIDAD        PRECIO         SUBTOTAL
--------------------------------------------------------------
Arroz                       2 $       5,000 $         10,000
Leche                       1 $       4,000 $          4,000
Pan                         3 $       1,500 $          4,500
--------------------------------------------------------------
TOTAL GENERAL                                $         18,500
==============================================================
```

Conceptos clave: Listas, tuplas, bucles for, acumuladores, multiplicación y alineación.

8. Generador de nombre de usuario

Requerimiento de la actividad: Crea un usuario con la primera letra del nombre, el apellido sin espacios internos y la cantidad total de letras.

Archivo: generador.py

```
nombre   = input("Primer nombre: ").strip().lower()
apellido = input("Primer apellido: ").strip().lower()

# Quitar espacios internos por si escriben nombre compuesto ("Juan Carlos")
nombre_limpio   = nombre.replace(" ", "")
apellido_limpio = apellido.replace(" ", "")

# Validación básica: ambos campos deben tener solo letras y no estar vacíos
if not nombre_limpio or not apellido_limpio:
    print("Error: Debe ingresar nombre y apellido.")
elif not nombre_limpio.isalpha() or not apellido_limpio.isalpha():
    print("Error: Solo se permiten letras (sin números ni símbolos).")
else:
    total_letras = len(nombre_limpio) + len(apellido_limpio)
    usuario = nombre_limpio[0] + apellido_limpio + str(total_letras)

    print("\n--- USUARIO GENERADO ---")
    print(f"Nombre completo: {nombre.capitalize()} {apellido.capitalize()}")
    print(f"Usuario:         {usuario}")
```

Cómo funciona: Pasa el texto a minúsculas, elimina espacios internos y rechaza campos vacíos o caracteres que no sean letras.

Cómo ejecutar: python generador.py

Datos de ejemplo: Nombre: Ana María; apellido: Pérez Gómez

Salida de referencia

```
--- USUARIO GENERADO ---
Nombre completo: Ana maría Pérez gómez
Usuario:         apérezgómez18
```

Conceptos clave: strip(), lower(), replace(), isalpha(), len(), indexación y condicionales.

9. Cálculo y división de propina

Requerimiento de la actividad: Calcula el valor de la propina, el total de la cuenta y cuánto paga cada persona.

Archivo: propina.py

```
# propuesto3.py

total_cuenta = int(input("Total de la cuenta: "))
propina_pct  = int(input("Porcentaje de propina a dejar (%): "))
personas     = int(input("Número de personas: "))

# Validaciones
if personas &lt;= 0:
    print("Error: El número de personas debe ser al menos 1.")
elif total_cuenta &lt; 0 or propina_pct &lt; 0:
    print("Error: El monto y el porcentaje no pueden ser negativos.")
else:
    propina       = total_cuenta * (propina_pct / 100)
    total_general = total_cuenta + propina
    por_persona   = total_general / personas

    print("\n--- RESUMEN DE LA CUENTA ---")
    print(f"Total cuenta:       ${total_cuenta:&gt;12,.0f}")
    print(f"Propina ({propina_pct:.0f}%):      ${propina:&gt;12,.0f}")
    print(f"Total general:      ${total_general:&gt;12,.0f}")
    print(f"Personas:           {personas:&gt;13}")
    print(f"Cada persona paga:  ${por_persona:&gt;12,.0f}")
```

Cómo funciona: Aplica el porcentaje al total, suma la propina a la cuenta y divide entre las personas. Valida el número de personas y los valores negativos.

Cómo ejecutar: python propina.py

Datos de ejemplo: Cuenta: 80000; propina: 10%; personas: 4

Salida de referencia

```
--- RESUMEN DE LA CUENTA ---
Total cuenta:       $      80,000
Propina (10%):      $       8,000
Total general:      $      88,000
Personas:                       4
Cada persona paga:  $      22,000
```

Conceptos clave: int, porcentajes, operaciones aritméticas, división y validación.

10. Validación básica de correo electrónico

Requerimiento de la actividad: Comprueba que el texto contenga @, tenga un punto después de ese símbolo y termine en .com o .edu.co.

Archivo: validador.py

```
correo = input("Ingrese su correo electrónico: ").strip().lower()

# Validaciones (cada una devuelve True/False)
tiene_arroba  = "@" in correo
termina_com   = correo.endswith(".com")
termina_edu   = correo.endswith(".edu.co")
tiene_punto   = "." in correo.split("@")[-1]  # hay un punto después del @?

if tiene_arroba and tiene_punto and (termina_com or termina_edu):
    print(f"✓ El correo '{correo}' tiene un formato VÁLIDO.")
else:
    print(f"✗ El correo '{correo}' tiene un formato INVÁLIDO.")
    if not tiene_arroba:
        print("   - Falta el símbolo '@'.")
    if not tiene_punto:
        print("   - No hay un punto después del '@'.")
    if not (termina_com or termina_edu):
        print("   - Debe terminar en '.com' o '.edu.co'.")
```

Cómo funciona: Usa endswith() para revisar el dominio final y split() para inspeccionar la parte posterior al @. Es una validación didáctica, no una comprobación exhaustiva de direcciones.

Cómo ejecutar: python validador.py

Datos de ejemplo: ana@example.com

Salida de referencia

```
El correo 'ana@example.com' tiene un formato VÁLIDO.
(El programa antepone un símbolo de confirmación.)
```

Conceptos clave: strip(), lower(), in, endswith(), split(), booleanos y operadores lógicos.